Artefactos de Código de Guardián Escolar

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Tecnólogo en Análisis y Desarrollo de Software

Ficha: 3145556

SENA — Sede La Industria, La Empresa y los Servicios

Neiva, Huila, Colombia

Octubre de 2026

Artefactos de Código de Guardián Escolar

1. Introducción

Guardian Escolar es una plataforma de gestión y seguimiento del transporte escolar. Su propósito es apoyar la administración de estudiantes, acudientes, conductores, vehículos, colegios y rutas, así como facilitar funcionalidades relacionadas con la autenticación, el seguimiento GPS, el abordaje y las notificaciones.

El *workspace* del proyecto contiene aplicaciones cliente, microservicios, una API Gateway, mecanismos de mensajería, scripts de base de datos y documentación técnica.

Este documento presenta un inventario de los artefactos técnicos identificados en los archivos del proyecto. Se consideran artefactos de código los archivos fuente, componentes, controladores, servicios, modelos, configuraciones, scripts SQL, pruebas automatizadas y otros recursos utilizados para desarrollar y mantener el software.

La expresión «implementado en código» indica que existe un artefacto con lógica o configuración relacionada con una funcionalidad. No significa necesariamente que dicha funcionalidad haya sido ejecutada, probada o validada integralmente.

2. Objetivo y alcance del inventario

2.1. Objetivo general

Documentar los artefactos técnicos disponibles de Guardian Escolar, describiendo su organización, tecnologías, responsabilidades y estado de verificación, con el propósito de facilitar el mantenimiento del software y apoyar la elaboración de los entregables técnicos del proyecto ADSO.

2.2. Objetivos específicos

- Identificar los repositorios y módulos que componen el proyecto.

- Describir las tecnologías, lenguajes, *frameworks* y dependencias declaradas.
- Identificar los principales archivos y componentes de las aplicaciones web y móvil.
- Documentar los microservicios, sus responsabilidades y las rutas API identificadas.
- Describir los artefactos de base de datos, infraestructura, mensajería y configuración.
- Identificar las pruebas automatizadas existentes y diferenciar su existencia de su ejecución.
- Registrar inconsistencias, duplicidades y riesgos técnicos que requieren seguimiento.
- Establecer una relación entre las necesidades del sistema y los artefactos que aportan evidencia de su implementación.

2.3. Alcance

El inventario comprende 15 repositorios Git identificados en el *workspace*, que abarcan los siguientes componentes:

- Aplicación web Angular.
- Aplicación móvil desarrollada con Expo y React Native.
- Servicios de negocio desarrollados con .NET.
- Servicios transversales de identidad (IAM) y GPS desarrollados con Java y Spring Boot.
- API Gateway Kong.
- Infraestructura de mensajería Apache Kafka.
- Scripts de SQL Server y migraciones Liquibase.
- Documentación técnica, diagramas y pruebas automatizadas.

El alcance no incluye la certificación del funcionamiento integral del sistema. No se ejecutaron compilaciones, migraciones, pruebas completas de integración ni despliegues durante la auditoría descrita.

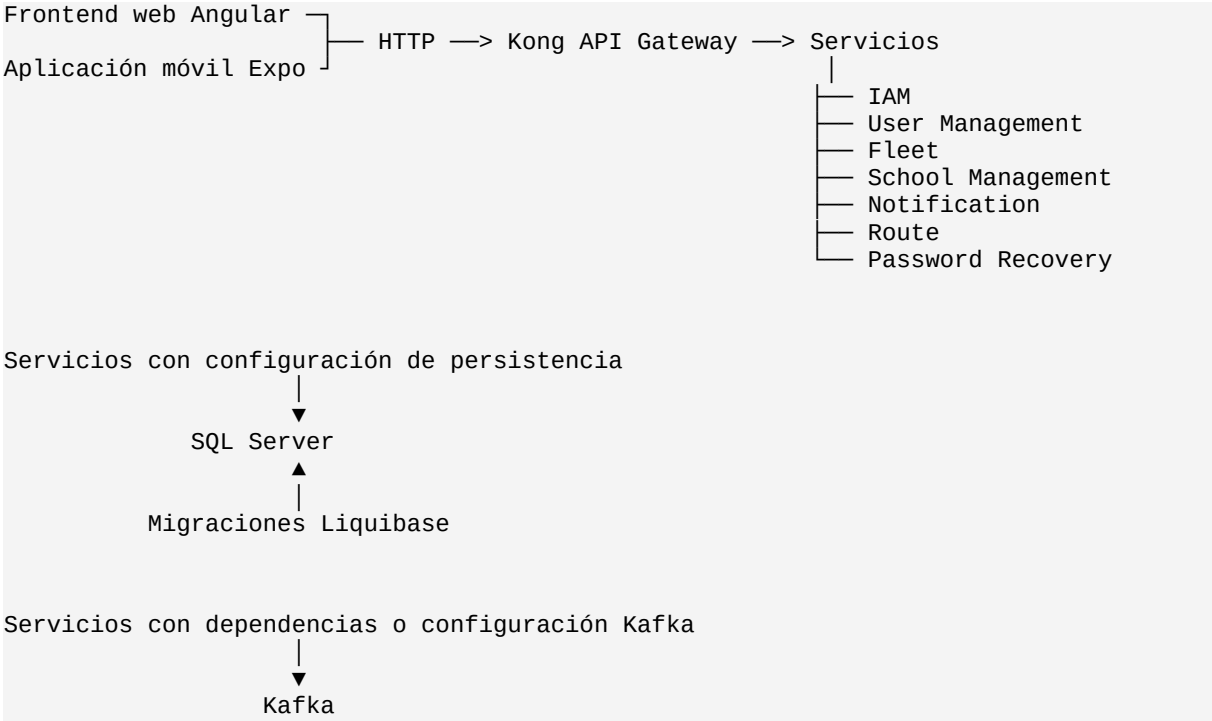
3. Arquitectura general del sistema

Guardian Escolar presenta una arquitectura distribuida que integra aplicaciones cliente, servicios *backend*, persistencia de datos y mecanismos de comunicación.

La Figura 1 resume las relaciones identificadas en el código y la configuración. Kong contiene configuración de enrutamiento hacia determinados servicios internos. SQL Server y Liquibase aparecen en los archivos de infraestructura y migración. Kafka dispone de configuración de *broker* y declaración de *topics*.

Figura 1

Arquitectura general declarada de Guardian Escolar



Nota. El diagrama representa una vista general de la arquitectura declarada. No implica que todas las conexiones se encuentren operativas ni que todos los servicios se comuniquen directamente entre sí.

El GPS se considera un servicio transversal separado. No se confirmó su publicación mediante Kong, ya que el archivo `kong.yml` revisado no declara un servicio GPS.

Estado de la arquitectura: existen artefactos de código y configuración que describen la arquitectura; su funcionamiento integral no ha sido validado mediante pruebas de ejecución.

4. Inventario de repositorios y módulos

La Tabla 1 resume los repositorios identificados y su responsabilidad observable. La clasificación se basa en los artefactos identificados; la existencia de un repositorio no demuestra por sí misma que el servicio correspondiente se encuentre desplegado o disponible.

Tabla 1

Repositorios identificados en el workspace

Repositorio	Tipo	Responsabilidad identificada	Estado
`broker-kafka`	Infraestructura	Configuración de Kafka y creación de topics	Implementado en configuración
`Guardian Escolar`	Monorepo	Frontend web, aplicación móvil, GPS histórico y documentos	Implementado en código
`sg-db`	Base de datos	SQL Server, Liquibase, DDL y DML	Implementado en scripts
`sg-documentation`	Documentación	Requisitos, diseño, propuesta técnica, pruebas, manuales y ACPM	Diseñado/documentado
`sg-ms-api-gateway`	Transversal	Configuración de Kong y rutas API	Implementado en configuración
`sg-ms-configuration`	Transversal	Documentación README; sin implementación observable en la revisión	Parcialmente implementado
`sg-ms-exceptional`	Transversal	Estructura inicial de un servicio de excepciones en .NET	Parcialmente implementado
`sg-ms-fleet`	Negocio	Gestión de buses, vehículos, conductores y dispositivos GPS asociados	Implementado en código
`sg-ms-forgot-information`	Transversal	Recuperación de acceso y actualización de información	Implementado en código
`sg-ms-gps`	Transversal	Consulta y pruebas de posiciones GPS	Implementado en código
`sg-ms-iam`	Transversal	Autenticación, perfiles, JWT y términos	Implementado en código

Repositorio	Tipo	Responsabilidad identificada	Estado
`sg-ms-notification`	Negocio/ transversal	Escaneo, tokens de dispositivos y notificaciones	Implementado en código
`sg-ms-route`	Negocio	Gestión de rutas, paradas, asignaciones y viajes	Implementado en código
`sg-ms-school-management`	Negocio	Gestión de colegios y sedes	Implementado en código
`sg-ms-user-management`	Negocio	Gestión de personas, familias, estudiantes, acudientes y conductores	Implementado en código

5. Tecnologías, lenguajes, frameworks y dependencias

Las tecnologías indicadas en la Tabla 2 corresponden a las versiones declaradas en los archivos de configuración revisados. No representan necesariamente las versiones efectivamente instaladas o ejecutadas en un entorno operativo.

También se identifican dependencias y mecanismos relacionados con JWT, BCrypt, gRPC, Kafka y Expo SecureStore. Su utilización efectiva debe comprobarse mediante la revisión del código específico y, cuando corresponda, mediante pruebas.

La documentación histórica que menciona PostgreSQL, MySQL o Flutter debe tratarse como documentación de etapas anteriores. La configuración actual identificada utiliza SQL Server, Angular y Expo/React Native. Los documentos históricos deben conservarse como antecedentes, pero no deben presentarse como descripción definitiva de la arquitectura vigente.

Tabla 2

Tecnologías y versiones declaradas

Tecnología	Versión declarada	Archivo de evidencia	Estado
Angular	`^21.2.9`	`Guardian Escolar/dvlp-front/dvlp-web/package.json`	Declarada en manifest
TypeScript web	`~5.9.2`	`dvlp-web/package.json`	Declarada en manifest
Expo	`~57.0.27`	`dvlp-movil/guardian-escolar/package.json`	Declarada en manifest
React móvil	`19.2.3`	`dvlp-movil/guardian-escolar/package.json`	Declarada en manifest

Tecnología	Versión declarada	Archivo de evidencia	Estado
React Native	`0.86.3`	`dvlp-movil/guardian-escolar/package.json`	Declarada en manifest
TypeScript móvil	`~6.0.3`	`dvlp-movil/guardian-escolar/package.json`	Declarada en manifest
.NET	`net10.0`	Archivos `*.csproj` de los microservicios	Declarado en manifests
Java	`21`	POM de IAM, GPS y GPS histórico	Declarado en manifests
Spring Boot	`4.1.1`	POM padre de IAM y GPS	Declarado en manifests
SQL Server	Imagen `2022-latest`	`sg-db/docker-compose.yaml`	Declarado en configuración
Apache Kafka	Imagen `apache/kafka:4.1.0`	`broker-kafka/docker-compose.yml`	Declarado en configuración
Kong	Imagen `kong:3.9`	`sg-ms-api-gateway/docker-compose.yml`	Declarado en configuración
Liquibase	Imagen `liquibase:4.29`	`sg-db/docker-compose.yaml`	Declarado en configuración

6. Estructura de carpetas y componentes relevantes

La organización del código permite distinguir los componentes de interfaz, la lógica de negocio, la infraestructura y los recursos de persistencia. En la Tabla 3, las rutas internas deben interpretarse dentro del repositorio correspondiente, y los comodines como `**` representan grupos de archivos y no una ruta individual.

Tabla 3

Estructura de carpetas y componentes relevantes

Repositorio o módulo	Ruta interna	Artefactos principales	Estado
Web	`dvlp-web/src/app/core`	Servicios HTTP, autenticación y utilidades	Implementado en código
Web	`dvlp-web/src/app/features`	Pantallas públicas, perfil, administración y superadministración	Implementado en código
Móvil	`guardian-escolar/src/core`	Servicios, configuración, validación y hooks	Implementado en código

Repositorio o módulo	Ruta interna	Artefactos principales	Estado
Móvil	`guardian-escolar/src/features`	Login, perfil, inicio, QR, GPS y navegación	Implementado en código
Servicios .NET	`src/<servicio>.Api`	Controladores, aplicación, infraestructura y entidades	Implementado en código
Servicios .NET	`tests/<servicio>.Tests`	Pruebas automatizadas	Pruebas existentes
Servicios Java	`src/main/java`	Dominio, DTO, casos de uso e infraestructura	Implementado en código
Base de datos	`01-ddl`, `02-dml`, `changelog`	Esquemas, tablas, claves foráneas, índices y datos iniciales	Implementado en scripts
Gateway	`kong/kong.yml`	Servicios, rutas, CORS y limitación de solicitudes	Implementado en configuración

7. Artefactos del frontend web Angular

La aplicación web utiliza Angular y organiza sus componentes en áreas funcionales, servicios compartidos y elementos de infraestructura. Las carpetas y archivos identificados en la Tabla 4 permiten documentar la estructura del frontend; la disponibilidad de pruebas no demuestra que las pantallas o los servicios hayan superado una ejecución de pruebas.

Tabla 4

Artefactos del frontend web Angular

Artefacto	Repositorio y ruta relativa	Responsabilidad	Estado
Punto de entrada	`Guardian Escolar/dvlp-front/dvlp-web/src/main.ts`	Arranque de Angular mediante `bootstrapApplication`	Implementado en código
Aplicación raíz	`dvlp-web/src/app/app.ts`	Componente raíz y configuración de navegación	Implementado en código
Servicios núcleo	`dvlp-web/src/app/core/services/**`	Comunicación HTTP y servicios de dominio	Implementado en código
Interceptores y manejo HTTP	`dvlp-web/src/app/core/http/**`	Manejo de solicitudes y respuestas HTTP	Implementado en código
Administración	`dvlp-web/src/app/features/admin/**`	Funcionalidades de usuarios, buses, rutas y paradas	Implementado en código
Superadministración	`dvlp-web/src/app/features/superadmin/**`	Gestión de colegios y administradores	Implementado en código

Artefacto	Repositorio y ruta relativa	Responsabilidad	Estado
Perfil	`dvlp-web/src/app/features/profile/**`	Funcionalidades de contraseña, correo, teléfono e información del perfil	Implementado en código
Pruebas Angular	`dvlp-web/src/**/*.spec.ts`	Pruebas de servicios, vistas y componentes	Existentes; no ejecutadas

8. Artefactos de la aplicación móvil React Native/Expo

La aplicación móvil se desarrolla con Expo y React Native. Sus artefactos incluyen la inicialización de la aplicación, navegación, autenticación, configuración y pantallas funcionales. Durante la auditoría descrita no se ejecutaron pruebas de la aplicación móvil; por tanto, la presencia de los artefactos de la Tabla 5 no constituye evidencia de que la autenticación, la navegación, el GPS o la comunicación con el *backend* funcionen integralmente.

Tabla 5

Artefactos de la aplicación móvil React Native/Expo

Artefacto	Repositorio y ruta relativa	Responsabilidad	Estado
Punto de entrada	`Guardian Escolar/dvlp-front/dvlp-movil/guardian-escolar/index.js`	Registro de la aplicación mediante `registerRootComponent`	Implementado en código
Aplicación raíz	`guardian-escolar/App.js`	Navegación y proveedores globales	Implementado en código
Login	`src/features/auth/views/Login/Login.jsx`	Formulario de correo, contraseña y aceptación de términos	Implementado en código
Autenticación	`src/core/services/authService.js`	Solicitudes de login, refresh, logout y gestión de sesión	Implementado en código
Roles	`src/core/config/roles/roleConfig.js`	Mapeo de roles Student, Driver y Parent	Implementado en código
GPS	`src/core/config/gps.js`	Configuración de URL, IMEI e intervalo de consulta	Implementado en código
Perfil	`src/features/profile/**`	Datos personales, familia, seguridad, privacidad y apariencia	Implementado en código
Pruebas	`src/core/services/authService.test.mjs`, `scripts/*.test.mjs`	Pruebas de autenticación y scripts de red	Existentes; no ejecutadas

9. Artefactos de los microservicios .NET

Los servicios .NET contienen controladores y componentes de *backend* orientados a la gestión de flota, rutas, colegios, usuarios, recuperación de información y notificaciones. Los controladores de la Tabla 6 permiten identificar las rutas y operaciones declaradas; sin embargo, la existencia de métodos CRUD no demuestra que todos los casos de uso estén completos, que las reglas de negocio sean correctas o que los servicios estén disponibles en ejecución.

Tabla 6

Controladores y rutas de los microservicios .NET

Servicio	**Controladores identificados**	**Rutas declaradas por controlador**	**Estado**
Fleet	`BusController.cs`, `GpsDeviceController.cs`, `VehicleTypeController.cs`	`api/buses`, `api/gps-devices`, `api/vehicle-types`	Implementado en código
Route	`RouteController.cs`, `StopController.cs`, `TripController.cs`, `RouteAssignmentController.cs`, `CityController.cs`	`api/routes`, `api/stops`, `api/trips`, `api/cities`	Implementado en código
School Management	`SchoolController.cs`	`api/v1/schools`	Implementado en código
User Management	`AdminController.cs`, `DriverController.cs`, `ParentController.cs`, `StudentController.cs`, `FamilyController.cs`, `DriverLicenseController.cs`	`api/admins`, `api/drivers`, `api/parents`, `api/students`, `api/families`, `api/driver-licenses`	Implementado en código
Forgot Information	Controladores Password, EmailChange y PhoneChange	`api/v1/password`, `api/v1/email/change`, `api/v1/phone/change`	Implementado en código
Notification	`NotificationController.cs`	`api/v1/notifications`	Implementado en código
Exceptional	`ms-exceptional.Api.csproj` y estructura inicial del servicio	Catálogo funcional pendiente de verificación	Parcialmente implementado

10. Artefactos de los microservicios Java/Spring Boot

Java y Spring Boot se utilizan en los servicios de identidad y GPS identificados en el *workspace*. En la Tabla 7, las rutas que contienen `...` son referencias abreviadas y no rutas completas; deben sustituirse por las rutas reales si se requiere trazabilidad exacta a nivel de archivo.

Tabla 7

Artefactos de los microservicios Java/Spring Boot

Servicio	Artefacto	Ruta relativa	Responsabilidad	Estado
IAM	Controlador de autenticación	`sg-ms-iam/src/main/java/.../infrastructure/web/AuthController.java`	Login, refresh, logout, perfil y términos	Implementado en código
IAM	Caso de uso de login	`.../application/usecase/LoginService.java`	Búsqueda por correo, BCrypt, estado del perfil y JWT	Implementado en código
IAM	Persistencia	`.../ProfileJpaRepository.java`	Consultas relacionadas con perfil, persona, rol, sede y colegio	Implementado en código
IAM	Seguridad	`.../config/SecurityConfig.java`	Configuración de filtros y rutas públicas o protegidas	Implementado en código
GPS	Dispositivo	`sg-ms-gps/src/main/java/gps_backend/adapters/in/rest/GpsDeviceController.java`	Consulta de dispositivo y conexión	Implementado en código
GPS	Ubicación	`.../GpsLocationController.java`	Consulta de última ubicación	Implementado en código
GPS	Pruebas de posición	`.../GpsTestController.java`	Registro de posiciones de prueba	Implementado en código
GPS histórico	Aplicación y controladores	`Guardian Escolar/gps-backend/gps-backend/**`	Segunda implementación GPS identificada	Implementado en código

11. API Gateway Kong y contratos API

Kong funciona como puerta de entrada para las rutas declaradas en su configuración. El archivo principal identificado es `sg-ms-api-gateway/kong/kong.yml`.

La compatibilidad completa entre Kong y los controladores no se verificó mediante solicitudes reales. Se identificó que Kong declara una ruta para `/api/v1/password`, pero no se observaron rutas equivalentes para `/api/v1/email/change` ni `/api/v1/phone/change`, aunque existen controladores asociados a dichas funcionalidades en el servicio Forgot Information. Este hallazgo (H-02) debe revisarse para determinar si las rutas faltantes se publican mediante otra configuración o si se requiere ampliar el enrutamiento del Gateway.

Tabla 8

Servicios declarados en Kong y estado de compatibilidad

Servicio Kong	Prefijo externo	`strip_path`	URL upstream declarada	Estado de compatibilidad
IAM	`/api/v1/auth`	`false`	`http://ms-iam:8080`	Compatible por ruta declarada; sin prueba de ejecución
User Management	`/user`	`true`	`http://ms-user-management:8080`	Pendiente verificar llamadas de los clientes
Fleet	`/fleet`	`true`	`http://ms-fleet:8080`	Compatible en teoría con `/fleet/api/buses` hacia `/api/buses`; no ejecutado
School Management	`/school-management`	`true`	`http://ms-school-management:8080`	Compatible en teoría; no ejecutado
Notification	`/notification`	`true`	`http://ms-notification:8080`	Compatibilidad completa pendiente de prueba
Route	`/route`	`true`	`http://ms-route:8080`	Compatible en teoría; no ejecutado

Servicio Kong	Prefijo externo	strip_path	URL upstream declarada	Estado de compatibilidad
Forgot Information	/api/v1/password	false	http://ms-forgot-information:8080	Compatible únicamente con las rutas de contraseña identificadas; no se observaron rutas para /api/v1/email/change ni /api/v1/phone/change (H-02)

12. Base de datos, SQL y Liquibase

La configuración revisada define SQL Server como motor de persistencia y utiliza Liquibase para organizar las migraciones. Los scripts se encuentran distribuidos por dominios y contienen definiciones de esquemas, tablas, relaciones y datos iniciales.

12.1. Esquemas identificados

Los nombres de la Tabla 9 se presentan según la información de DDL recogida en la auditoría, con la grafía registrada en ella (por ejemplo, DriverAssignemts); deben conservarse exactamente como aparecen en los scripts originales.

Tabla 9

Esquemas y tablas identificados en el DDL

Esquema	Tablas identificadas en DDL
Iam	Role, Permissions, PermissionsRole, Profile, SessionProfile, TermsAcceptance
UserManagement	Person, Family, FamilyMember, DriverLicense
School	School, SchoolCampus, SchoolAdmin
Fleet	Brand, Model, Bus, DriverAssignemts
Route	Route, Stop, RouteStop, RouteBusAssignments, RouteStudentAssignments, RouteSchedule, RouteExecution
Gps	GpsDevice, GpsLocation
Notification	AlertType, Alert, AlertRecipient, Boarding, DeviceToken
Geographic	City
Settings	SecuritySettings, PasswordPolicies

12.2. Artefactos de persistencia

No se comprobó que las migraciones de la Tabla 10 hayan sido aplicadas en una instancia real de SQL Server; la existencia de scripts no demuestra que la base de datos esté creada, actualizada o disponible. Los archivos PostgreSQL ubicados en `Guardian Escolar/docs/database/` se consideran documentación histórica o legada, y no describen por sí solos la configuración actual basada en SQL Server y Liquibase.

Tabla 10

Artefactos de persistencia

Artefacto	Ruta relativa	Estado
Orquestación SQL/Liquibase	`sg-db/docker-compose.yaml`	Implementado en configuración
Migraciones IAM	`sg-db/ms-iam-db/**/changelog.yaml`	Implementado en scripts
Migraciones User Management	`sg-db/ms-user-management-db/**/changelog.yaml`	Implementado en scripts
Relaciones entre esquemas	`sg-db/cross-schema-fks/changelog.yaml`	Implementado en scripts
DDL y DML por dominio	`sg-db/ms-*-db/**`	Implementado en scripts

13. Kafka, eventos y comunicación

El repositorio `broker-kafka` contiene configuración para el *broker* y la declaración de *topics* que se utilizarían para la comunicación basada en eventos.

13.1. Topics declarados

- `student.created`
- `driver.created`
- `admin.created`
- `parent.created`
- `profile.created`

- `student.scanned`

13.2. Artefactos de Kafka

No se comprobó en ejecución la publicación o el consumo de mensajes, el tratamiento de errores, los reintentos, el orden de los eventos ni la recuperación ante fallos. Por esta razón, los *topics* deben considerarse artefactos configurados, no evidencia suficiente de una comunicación asíncrona operativa.

Tabla 11

Artefactos de Kafka

Artefacto	Ruta relativa	Estado
Broker	`broker-kafka/docker-compose.yml`	Implementado en configuración
Creación de topics	`broker-kafka/init/create-topics.sh`	Implementado en scripts
Catálogo de topics	`broker-kafka/init/topics.conf`	Implementado en configuración
Dependencias Kafka	IAM, Route, User Management y Notification	Declaradas en manifests o código; flujo no ejecutado

14. Backend GPS y posiciones

Se identificaron dos implementaciones GPS en el *workspace*. La duplicidad de implementaciones es un hallazgo confirmado en el inventario (H-01). Sin embargo, no se puede determinar cuál es la implementación canónica, cuál está desplegada o si una debe reemplazarse sin una decisión documentada del equipo y evidencia del despliegue utilizado.

La validación del GPS requiere comprobar la comunicación entre el dispositivo o simulador, el *backend*, la persistencia de posiciones y la aplicación cliente.

Tabla 12

Implementaciones GPS identificadas

Implementación	Ubicación	Evidencia identificada	Estado
Microservicio GPS	`sg-ms-gps`	POM Java/Spring Boot, controladores REST y documentación de arquitectura	Implementado en código
GPS histórico	`Guardian Escolar/gps-backend/gps-backend`	POM Java/Spring Boot y controladores REST	Implementado en código

15. Infraestructura, Docker y configuración

Los archivos Docker Compose y las configuraciones asociadas describen los componentes que se pretenden ejecutar y sus dependencias. No se verificó que todos los archivos Compose puedan ejecutarse conjuntamente ni que las dependencias externas estén disponibles.

Los valores sensibles de los archivos `.env` no deben incorporarse a la documentación. Las variables necesarias para instalar o ejecutar el sistema deben documentarse por nombre, finalidad y ejemplo seguro, sin publicar contraseñas, *tokens* ni claves reales.

Tabla 13

Artefactos de infraestructura y configuración

Artefacto	Ruta	Responsabilidad	Estado
Kafka Compose	`broker-kafka/docker-compose.yml`	Definición del broker y volumen	Implementado en configuración
Base de datos Compose	`sg-db/docker-compose.yaml`	SQL Server, inicialización y Liquibase	Implementado en configuración
Gateway Compose	`sg-ms-api-gateway/docker-compose.yml`	Configuración del contenedor Kong y puertos	Implementado en configuración
Gateway declarativo	`sg-ms-api-gateway/kong/kong.yml`	Rutas, CORS y limitación de solicitudes	Implementado en configuración
Compose por servicio	`sg-ms-*/docker-compose.yml`	Configuración de contenedores de microservicios	Implementado en configuración
Variables de ejemplo	`**/.env.example`	Declaración de variables requeridas	Implementado en configuración

Artefacto	Ruta	Responsabilidad	Estado
Variables locales	`**/.env`	Valores de desarrollo	No se incluyen valores; revisión pendiente

16. Pruebas automatizadas existentes

La auditoría identificó archivos de pruebas en varios repositorios. No obstante, no se ejecutaron las pruebas y no se dispone de resultados de cobertura ni de integración continua revisados. La existencia de pruebas es una evidencia útil para mantenimiento, pero no permite afirmar que el software las supere. Para validar este aspecto se deben registrar los comandos ejecutados, las versiones de herramientas, los resultados, los fallos encontrados y, si está disponible, el porcentaje de cobertura.

Tabla 14

Pruebas automatizadas existentes y estado de validación

Área	Evidencia de código	Pruebas identificadas	Validación ejecutada
Fleet	Servicios y controladores Fleet	`tests/ms-fleet.Tests/**`	No
Route	Servicios y controladores Route	`tests/ms-route.Tests/**`	No
User Management	Servicios y controladores User Management	`tests/ms-user-management.Tests/**`	No
School Management	Controladores del servicio School	`tests/ms-school-management.Tests/**`	No
Forgot Information	Controladores de contraseña, correo y teléfono	`tests/ms-forgot-information.Tests/**`	No
IAM	Casos de uso y filtros	`sg-ms-iam/src/test/java/**`	No
Web	Servicios, componentes y pantallas	`dvlp-web/src/**/*.spec.ts`	No
Móvil	Autenticación y scripts	`guardian-escolar/**/*.test.mjs`	No

17. Documentación técnica y diagramas

Además del código fuente, el proyecto cuenta con documentación de requisitos, diseño, implementación y mantenimiento. La documentación debe mantenerse alineada con el código vigente. Cuando existan diferencias entre documentos históricos y la implementación actual, estas deben registrarse y corregirse mediante un proceso controlado de actualización documental.

Tabla 15

Documentación técnica y diagramas

Ubicación	Tipo de artefacto	Estado documental
`sg-documentation/01-Informe-Especificacion-Requisitos-Software`	Especificación de requisitos	Diseñado/documentado
`sg-documentation/04-Diseno-Software`	Diseño técnico	Documentado; requiere contraste con el código
`sg-documentation/Manuales`	Manual técnico y manual de instalación	Documentado; instalación no validada
`Guardian Escolar/docs/SRS`	SRS histórico y fuentes de requisitos	Histórico; no representa por sí solo la arquitectura vigente
`Guardian Escolar/docs/diagrams`	UML, Draw.io, PUML y Gantt	Diseñado/documentado
`Guardian Escolar/docs/database`	MER, DBML y scripts históricos	Parcialmente vigente; requiere contraste con Liquibase
`sg-ms-gps/ARCHITECTURE.md`	Diseño arquitectónico GPS	Diseñado/documentado

18. Matriz de trazabilidad entre necesidades y artefactos

La Tabla 16 relaciona las necesidades funcionales principales con los artefactos identificados. La columna de validación distingue entre la existencia del código y la ejecución de pruebas. Esta matriz constituye una primera trazabilidad técnica; para considerarla completa debe vincularse cada requisito con su identificador formal en la especificación, la ruta exacta del artefacto y el caso de prueba correspondiente.

Tabla 16

Matriz de trazabilidad entre necesidades y artefactos

Requisito o necesidad	Módulo	Evidencia de código	Pruebas existentes	Validación ejecutada	Estado
Gestión de usuarios	User Management	Controladores Admin, Driver, Parent, Student y Family	Sí, según inventario	No	Implementado en código
Autenticación y perfiles	IAM / móvil	`AuthController`, `LoginService`, `authService.js`	Sí, según inventario	No	Implementado en código
Gestión de colegios y sedes	School Management	`SchoolController.cs` y tablas School	Sí, según inventario	No	Implementado en código
Buses y conductores	Fleet	`BusController.cs` y entidades Fleet	Sí, según inventario	No	Implementado en código
Rutas, paradas y viajes	Route	Controladores Route, Stop, Trip y Assignment	Sí, según inventario	No	Implementado en código
QR y abordaje	Notification / móvil	Endpoint de escaneo, componentes QR y topic `student.scanned`	Prueba integrada pendiente de identificar	No	Parcialmente implementado
Posición GPS	GPS / móvil	Controladores GPS, `GpsLocation` y `gps.js`	No identificada	No	Parcialmente implementado
Notificaciones	Notification / Kafka	Controlador, tablas y topics	Integración pendiente	No	Parcialmente implementado
Recuperación de cuenta	Forgot Information	Controladores Password, EmailChange y PhoneChange	Sí, según inventario	No	Implementado en código
Gestión de términos	IAM / móvil	`TermsAcceptance`, endpoints y login móvil	Prueba específica pendiente	No	Implementado en código

19. Hallazgos, duplicidades e inconsistencias

Los hallazgos de la Tabla 17 se clasifican según la evidencia identificada. La clasificación «confirmado» significa que el hecho se observa en archivos revisados, no necesariamente que se haya reproducido durante la ejecución del sistema. Estos hallazgos sirven como insumo para el documento de acciones correctivas, preventivas y de mejoramiento del software; cada acción deberá contar con una prioridad, responsable, plazo, evidencia de cierre y criterio de aceptación.

Tabla 17

Hallazgos, duplicidades e inconsistencias identificados

ID	Hallazgo	Clasificación	Evidencia	Estado o acción requerida
H-01	Existen dos implementaciones GPS.	Confirmado en archivos	`sg-ms-gps` y `Guardian Escolar/gps-backend`	Definir y documentar la implementación canónica
H-02	Kong no declara rutas para cambio de correo ni teléfono en la configuración revisada.	Confirmado en configuración	`kong.yml` y controladores de Forgot Information	Verificar si existen rutas alternativas y completar el enrutamiento si corresponde
H-03	La consulta de autenticación revisada no filtra explícitamente el estado de persona y rol, aunque valida el estado del perfil.	Confirmado en código revisado	`LoginService.java` y `ProfileJpaRepository.java`	Revisar las reglas de negocio y los controles de autorización
H-04	El refresh token se incluye en la respuesta JSON y también se configura una cookie HttpOnly.	Confirmado en código revisado	`AuthController.java` y `LoginResponseDto`	Revisar la necesidad de ambas formas de entrega y los riesgos de exposición
H-05	No se validó integralmente la comunicación entre Gateway, servicios y pruebas.	Confirmado como limitación de auditoría	No se ejecutaron pruebas completas	Planificar y documentar pruebas de integración

ID	Hallazgo	Clasificación	Evidencia	Estado o acción requerida
H-06	La documentación histórica menciona PostgreSQL, MySQL y Flutter.	Confirmado documentalmente	SRS, informe técnico y documentos históricos de tecnologías	Etiquetar como legado y actualizar la documentación vigente
H-07	El DBML histórico incluye dominios de auditoría y excepciones que no se comprobaron como esquemas operativos completos.	Pendiente de confirmación integral	`docs/database/db-school-guardian.dbml` frente al DDL actual	Comparar el modelo con los scripts vigentes y los requisitos
H-08	Algunos puertos administrativos de Kong se publican en la configuración Compose.	Confirmado en configuración; riesgo potencial	`sg-ms-api-gateway/docker-compose.yml`	Revisar exposición y restringir los puertos según el entorno

20. Limitaciones de la auditoría

La auditoría presenta las siguientes limitaciones:

- No se dispuso de acceso operativo a Docker para inspeccionar los contenedores.
- No hubo acceso de solo lectura a una instancia de SQL Server.
- No se ejecutaron compilaciones, pruebas automatizadas ni migraciones.
- No se verificaron los flujos completos de QR, Kafka, notificaciones, correo, SMS o GPS físico.
- No se confirmó que todas las rutas del Gateway correspondan a las llamadas realizadas por las aplicaciones cliente.
- No se verificó la cobertura de pruebas ni la ejecución de *pipelines* de integración continua.
- No se determinó cuál de las dos implementaciones GPS es la canónica.
- No se incluyeron valores sensibles de archivos `.env`, contraseñas, *tokens* ni claves privadas.

Por estas limitaciones, las conclusiones se restringen a los artefactos identificados en el *workspace* y no constituyen una certificación de funcionamiento del sistema.

21. Conclusiones

Guardian Escolar dispone de una base de artefactos técnicos amplia, compuesta por un frontend web Angular, una aplicación móvil Expo/React Native, servicios .NET, servicios Java/Spring Boot, una API Gateway Kong, mensajería Kafka, scripts de SQL Server y migraciones Liquibase.

El inventario permite identificar los principales componentes, su organización y las evidencias de código o configuración asociadas. También permite registrar áreas que requieren revisión, como la duplicidad del *backend* GPS, las rutas del Gateway para cambio de información, determinadas reglas de autenticación, la exposición de puertos administrativos y la ausencia de resultados de pruebas ejecutadas.

No obstante, la existencia de código y configuración no demuestra el funcionamiento integral del software. Para establecer ese estado se requiere ejecutar pruebas, verificar las migraciones en una instancia controlada y demostrar la comunicación entre clientes, Gateway, microservicios, Kafka, GPS y SQL Server.

En consecuencia, este documento debe considerarse un inventario técnico basado en la auditoría del *workspace*, sujeto a validación de ejecución y actualización controlada a medida que avance el proyecto.

22. Correcciones realizadas en la revisión documental

La revisión del inventario incorporó los siguientes ajustes:

- Se distinguieron las versiones declaradas en *manifests* de las versiones efectivamente ejecutadas.
- Se ajustó el diagrama de arquitectura para evitar afirmar conexiones no demostradas.
- Se separaron los servicios de negocio de los servicios transversales.

- Se incorporaron rutas de artefactos y referencias a archivos de configuración.
- Se diferenciaron las pruebas existentes de las pruebas ejecutadas.
- Se documentó la diferencia entre las rutas declaradas en Kong y los controladores de cambio de correo y teléfono.
- Se clasificaron los hallazgos según su evidencia y estado de verificación.
- Se registró la duplicidad del GPS sin decidir cuál implementación debe eliminarse.
- Se separó la documentación histórica de la descripción de la arquitectura actual.
- Se mantuvieron explícitas las limitaciones de la auditoría.

Estas correcciones corresponden a la revisión del documento. La verificación técnica definitiva de cada afirmación depende de consultar los archivos originales y ejecutar las pruebas correspondientes.

23. Datos no verificables

Durante la auditoría no fue posible verificar los siguientes aspectos:

- Versiones efectivas de dependencias y contenedores en ejecución.
- Estado y salud de los servicios desplegados.
- Aplicación real de las migraciones Liquibase.
- Estado de las tablas y datos de una instancia real de SQL Server.
- Compatibilidad completa de todos los clientes con las rutas del Gateway.
- Implementación GPS canónica.
- Cobertura de pruebas, resultados de CI/CD y despliegues.
- Funcionamiento integral de QR, Kafka, notificaciones y GPS.

Estos elementos deben permanecer identificados como pendientes hasta disponer de evidencia verificable.

24. Evidencias pendientes para validar el funcionamiento

Para complementar este inventario se recomienda recopilar las siguientes evidencias:

1. Resultados de compilación y pruebas automatizadas por repositorio.
2. Reportes de cobertura de pruebas, cuando estén disponibles.
3. Estado de los contenedores y registros de ejecución de los servicios.
4. Evidencia de migraciones Liquibase aplicadas en una base de datos controlada.
5. Consultas SQL de solo lectura que permitan comprobar esquemas y tablas.
6. Pruebas de integración entre Kong y los microservicios.
7. Prueba autorizada de autenticación desde la aplicación móvil.
8. Prueba controlada del flujo QR, publicación del evento y generación de notificación.
9. Prueba GPS simulada o física, documentando origen, recepción y persistencia de posiciones.
10. Decisión documentada del equipo sobre la implementación GPS canónica.

Cada evidencia debe indicar fecha, entorno, responsable, procedimiento y resultado, evitando incluir secretos o datos personales innecesarios.

25. Archivos consultados

La auditoría reporta la revisión de los siguientes archivos y grupos de archivos. Esta lista resume las referencias recogidas en la auditoría; para una trazabilidad completa se recomienda registrar la ruta exacta de cada archivo utilizado para respaldar cada hallazgo.

- `Guardian Escolar/dvlp-front/dvlp-web/package.json`
- `Guardian Escolar/dvlp-front/dvlp-movil/guardian-escolar/package.json`
- `sg-ms-iam/pom.xml`
- `sg-ms-gps/pom.xml`

- `Guardian Escolar/gps-backend/gps-backend/pom.xml`
- Archivos `*.csproj` de los microservicios .NET
- `sg-ms-api-gateway/kong/kong.yml`
- `sg-ms-api-gateway/docker-compose.yml`
- `sg-db/docker-compose.yaml`
- `broker-kafka/docker-compose.yml`
- `broker-kafka/init/topics.conf`
- Controladores, servicios, repositorios y filtros de los microservicios
- Archivos de pruebas automatizadas identificados en los repositorios
- Scripts DDL, DML y changelogs de Liquibase descritos en las secciones anteriores
- Documentación técnica y diagramas referenciados en el inventario